Layanan tugas akhir: pengajuan judul, penetapan pembimbing, bimbingan, seminar atau sidang, dan penilaian



Dosen Pengampu: Mardiyyah Hasnawi, S.Kom.,M.T.

Disusun Oleh: Kelompok I – 04

Annisa Rezky Ashriani 13020240332
Artika Milad Arhidayah 13020240049
Andi Naqia Aura WR 13020240279
B.Nadia Salsabila 13020240260

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS ILMU
KOMPUTER UNIVERSITAS MUSLIM INDONESIA
MAKASSAR
2026

Bab 1

Pendahuluan

Konteks dan ASR (Architectural Significant Requirements) (U1)

1.1 Ringkasan Layanan

Layanan Tugas Akhir merupakan salah satu layanan akademik pada platform *Smart Campus* Universitas Muslim Indonesia (UMI), yang termasuk dalam subsistem *Assignment & Assessment*. Layanan ini membantu mahasiswa dalam mengurus tugas akhir, mulai dari pengajuan judul proposal, penentuan dosen pembimbing, pengisian kegiatan bimbingan (*logbook*), sampai pemantauan perkembangan skripsi.

Proses lainnya meliputi pengumpulan dan pemeriksaan berkas persyaratan, pengaturan jadwal seminar dan sidang akhir, serta pencatatan nilai hasil penilaian. Dengan adanya layanan ini, mahasiswa, dosen, dan staf akademik dapat mengelola setiap tahapan tugas akhir dengan lebih mudah dan teratur.

1.2 Masalah Utama Layanan dan Bukti Lapangan

- **Masalah Utama:**

Berdasarkan hasil wawancara dengan lima responden, ditemukan beberapa kendala dalam pengelolaan Layanan Tugas Akhir. Permasalahan utama meliputi lambatnya respons layanan hingga berpotensi terjadi request timeout, serta kegagalan atau lamanya proses unggah draf skripsi, terutama pada dokumen berukuran besar dan memiliki banyak gambar. Peningkatan jumlah pengguna secara bersamaan saat pembukaan pendaftaran juga dapat menyebabkan layanan sulit diakses dan mengganggu proses pendaftaran seminar maupun sidang.

Selain itu, pencatatan bimbingan dan revisi belum terkelola dengan baik sehingga riwayat perkembangan tugas akhir terkadang sulit dipantau. Kendala lainnya mencakup informasi persyaratan sidang yang kurang jelas, penjadwalan yang berpotensi berbenturan, pemantauan kuota dosen pembimbing, serta pemeriksaan berkas mahasiswa yang masih harus dilakukan berulang kali.

Beberapa temuan tersebut diperoleh dari wawancara dengan responden sebagai berikut:

- **Bukti Lapangan (Berdasarkan Wawancara Responden):**

R1 Mahasiswa Akhir(25 Oktober 2026): Akses melambat saat pendaftaran ramai, notifikasi persetujuan logbook belum otomatis, dan unggah dokumen terkendala ukuran file.

R3 Dosen Pembimbing/Penguji(26 Oktober 2026): Pemeriksaan draf dan riwayat bimbingan belum terpusat; jadwal sidang berpotensi berbenturan dengan jadwal mengajar.

R5 Staf administrasi akademik (27 Oktober 2026): Berkas persyaratan yang tidak lengkap atau salah format menyebabkan pemeriksaan harus dilakukan berulang kali. Penggunaan ruangan sidang juga berpotensi berbenturan saat periode puncak.

1.3 Stakeholder Utama

Pihak-pihak yang memiliki peran dalam pengelolaan Layanan Tugas Akhir pada subsistem Assignment & Assessment Universitas Muslim Indonesia (UMI) adalah sebagai berikut:

1. Mahasiswa tingkat akhir Mengajukan judul, mengisi logbook, mengunggah dokumen, dan memantau status persetujuan tugas akhir**. Keep Informed**
2. Dosen pembimbing dan penguji Memeriksa judul dan dokumen, memberikan bimbingan, serta melakukan penilaian seminar dan sidang. **Key Player**
3. Ketua Program Studi Mengatur penugasan dosen pembimbing, memantau perkembangan mahasiswa, dan mengawasi hasil kelulusan. **Key Player**
4. Staf administrasi akademik Memeriksa persyaratan mahasiswa, mengatur jadwal dan ruangan sidang, serta mengelola data nilai. **Keep Informed**

1.4 Kebutuhan Signifikan Arsitektur (ASR)

Berdasarkan hasil analisis dan wawancara, terdapat tiga kebutuhan utama yang perlu diperhatikan dalam perancangan Layanan Tugas Akhir, yaitu:

1. Performance (Kinerja):Layanan harus mampu menangani hingga 500 pengguna secara bersamaan dengan target waktu muat halaman rata-rata kurang dari 3 detik.
2. Integrity & Auditability:Setiap aktivitas penting, seperti pengajuan judul, persetujuan bimbingan, dan pencatatan nilai harus tersimpan serta memiliki riwayat yang dapat ditelusuri.
3. Security (Keamanan):Dokumen tugas akhir dan nilai mahasiswa harus terlindungi serta hanya dapat diakses oleh pihak yang memiliki wewenang.

Ketiga kebutuhan tersebut diprioritaskan agar layanan tetap responsif, data terjaga, dan setiap proses tugas akhir dapat dipantau dengan baik.

1.5 Keterkaitan Quality Drivers dengan ASR

	Quality Driver	ASR Terkait	ASR Terkait Fokus Kebutuhan
1	Performance Efficiency	ASR-1	Performance Membantu layanan tetap cepat meskipun diakses banyak pengguna secara bersamaan.
2	Security & Integrity	ASR-2 dan ASR-3	Security dan Integrity & Auditability Menjaga data tugas akhir agar tidak diakses sembarang pihak dan memastikan setiap aktivitas penting tercatat.
3	Maintainability	Mendukung seluruh ASR	Memudahkan pengelolaan dan perbaikan layanan jika ada perubahan aturan akademik.

Ketiga Quality Drivers tersebut dipilih karena sesuai dengan kendala dan kebutuhan yang ditemukan dari hasil wawancara. Dengan begitu, layanan diharapkan lebih mudah digunakan, aman, dan tidak sulit diperbaiki ketika diperlukan.

Bab 2

Skenario Atribut Kualitas

Skenario Atribut Kualitas(U2)

Bagian ini merinci tiga skenario atribut kualitas berdasarkan karakteristik pendorong utama (*Performance Efficiency*, *Security*, dan *Maintainability*). Masing-masing skenario diuraikan ke dalam 6 komponen ATAM (*Architecture Tradeoff Analysis Method*), di mana Skenario 1 dirancang khusus untuk menjawab masalah utama terkait lonjakan trafik pendaftaran dan kendala unggah berkas draf skripsi.

2.1 Skenario 1: Performance Efficiency (Menjawab Masalah Utama)

- Sumber (*Source*): Mahasiswa tingkat akhir.

- Stimulus (*Stimulus*): Mengakses tombol "Daftar Sidang" dan mengunggah berkas draf skripsi berukuran besar (hingga 25 MB) secara serentak pada pukul 08.00 WITA.
- Artefak (*Artifact*): Modul Pendaftaran (*Assignment & Assessment*) dan *Message Queue Broker*.
- Lingkungan (*Environment*): Kondisi beban puncak (*peak traffic load* dengan 500 pengguna konkuren).
- Respons (*Response*): Sistem menampung permintaan masuk ke dalam antrean terpusat (*message queue*) dan memprosesnya secara asinkron di latar belakang (*background worker*) tanpa mengalami *crash* atau *race condition*.
- Ukuran Respons (*Response Measure*):
    - Waktu tanggap (*latency*) antarmuka tetap stabil di bawah 3 detik per permintaan.
    - Tingkat kegagalan sistem (*error rate*) sebesar 0% (seluruh antrean pendaftaran berhasil diproses).

2.3 Skenario 2: Security & Integrity (Keamanan Dokumen Draf Skripsi)

- Sumber (*Source*): Pengguna eksternal atau pihak tidak berwenang.
- Stimulus (*Stimulus*): Mencoba mengunduh berkas draf skripsi atau rekam nilai sidang mahasiswa lain melalui manipulasi tautan URL langsung (*direct object reference*).
- Artefak (*Artifact*): Modul Otorisasi dan Layanan *Cloud Object Storage*.
- Lingkungan (*Environment*): Operasional harian sistem.
- Respons (*Response*): Sistem memvalidasi token akses berbasis peran (*Role-Based Access Control / JWT*) dan langsung menolak permintaan unduh jika pengguna tidak memiliki hak akses yang sah.
- Ukuran Respons (*Response Measure*):
    - 100% upaya akses ilegal terdeteksi, diblokir, dan dicatat ke dalam log audit keamanan (*security audit log*).

2.3 Skenario 3: Maintainability (Kemudahan Pemeliharaan & Modifikasi Aturan)

- Sumber (*Source*): Administrator akademik / Fakultas.
- Stimulus (*Stimulus*): Melakukan perubahan pada parameter aturan minimum persyaratan SKS kelulusan tugas akhir di dalam kode sumber sistem.
- Artefak (*Artifact*): Lapisan Logika Bisnis (*Business Logic Layer*).
- Lingkungan (*Environment*): Tahap pemeliharaan sistem (*maintenance phase*).
- Respons (*Response*): Tim pengembang memperbarui modul aturan validasi secara terisolasi tanpa merusak modul tampilan antarmuka (*UI*) maupun struktur basis data dokumen.
- Ukuran Respons (*Response Measure*):
    - Waktu total implementasi perubahan kode dan pengujian ulang (*change implementation time*) selesai dalam waktu kurang dari 2 hari kerja.

Bab 3

TAKTIK DAN POLA ARSITEKTUR

3.1 Taktik Arsitektur(U3)

Bagian ini mendokumentasikan taktik arsitektur (*architectural tactics*) yang diterapkan untuk merespons ketiga skenario atribut kualitas pada subsistem Layanan Tugas Akhir (*Assignment & Assessment*). Setiap taktik dirancang untuk mencapai sasaran kualitas tertentu dengan tetap memperhitungkan kompromi (*trade-off*) yang diterima.

3.1.1 Taktik untuk Skenario 1: Performance Efficiency (Manajemen Lonjakan Trafik)

- **Taktik yang Diterapkan:**
    - *Manage Demand* (Menerapkan Antrean / *Message Queue Broker*).
    - *Defer Response* (Pemrosesan Latar Belakang / *Background Worker*).
- **Cara Kerja:** Ketika ribuan mahasiswa secara serentak menekan tombol pendaftaran sidang pada jam pembukaan kuota, sistem tidak memproses

seluruh transaksi secara langsung ke basis data utama (*synchronous*). Permintaan masuk (*request*) akan ditampung terlebih dahulu ke dalam antrean terpusat menggunakan *Message Broker* (seperti RabbitMQ atau Redis Queue). Selanjutnya, *Background Worker* akan mengambil dan memproses antrean tersebut secara bertahap dan terstruktur tanpa membebani *web server*.

- **Harga yang Dibayar (Trade-off):**
    - Kompleksitas infrastruktur meningkat karena memerlukan penambahan komponen server/layanan *message broker*.
    - Mahasiswa mengalami sedikit penundaan instan (*latency*) karena mendapatkan konfirmasi antrean terlebih dahulu alih-alih respons langsung seketika (*instant response*).

3.1.2 Taktik untuk Skenario 2: Security & Integrity (Proteksi Dokumen Draf Skripsi)

- **Taktik yang Diterapkan:**
    - *Authenticate Actor* & *Authorize Access* (Autentikasi Berbasis Token / JWT dan *Role-Based Access Control*).
- **Cara Kerja**: Setiap permintaan unduh atau akses terhadap berkas draf skripsi dan rekam nilai sidang wajib menyertakan token akses terenkripsi (*JSON Web Token / JWT*). Modul otorisasi akan memvalidasi identitas serta peran pengguna (*student, lecturer, admin*). Jika pengguna mencoba mengakses berkas di luar hak aksesnya melalui manipulasi URL (*direct object reference*), sistem akan secara otomatis menolak permintaan tersebut dan mencatatnya ke dalam log audit keamanan.
- **Harga yang Dibayar (Trade-off):**
    - Terjadi tambahan beban komputasi server (*overhead*) untuk proses verifikasi kriptografi dan dekripsi token pada setiap permintaan data.
    - Proses pengelolaan masa kedaluwarsa token (*token expiration*) memerlukan penanganan logika manajemen sesi yang lebih ketat di sisi klien dan server.

3.1.3 Taktik untuk Skenario 3: Maintainability (Kemudahan Pemeliharaan Aturan Akademik)

- **Taktik yang Diterapkan:**
    - *Split Module / Encapsulation* (Pemisahan Lapisan Logika Bisnis).
- **Cara Kerja:** Aturan validasi syarat SKS kelulusan dipisahkan secara tegas dari antarmuka pengguna (*UI*) dan struktur basis data dokumen. Logika bisnis ini dikemas ke dalam modul layanan terisolasi (*service layer*), sehingga ketika ada perubahan kebijakan akademik dari fakultas, pengembang hanya perlu memperbarui kode di dalam modul aturan tersebut tanpa menyentuh bagian tampilan atau penyimpanan data.
- Harga yang Dibayar (Trade-off):
    - Jumlah berkas kode program (*codebase*) dan struktur direktori proyek menjadi lebih banyak dan terfragmentasi.
    - Membutuhkan dokumentasi antarmuka internal yang disiplin agar komunikasi antar-modul tetap terkelola dengan baik.

3.2 Pola dan Gaya Arsitektur(U4)

Bagian ini mendokumentasikan perbedaan mendasar antara gaya arsitektur dan pola arsitektur, serta merinci dua pola dari kategori berbeda yang diterapkan pada perancangan subsistem Layanan Tugas Akhir (*Assignment & Assessment*).

3.2.1. Perbedaan Gaya Arsitektur (*Architectural Style*) dan Pola Arsitektur (*Architectural Pattern*)

- Gaya Arsitektur (*Architectural Style*): Merupakan bentuk atau kerangka pengorganisasian tingkat tinggi dari sebuah sistem secara keseluruhan yang mendefinisikan bentuk topologi globalnya (contoh: gaya berlapis/*layered style*, gaya klien-peladen/*client-server style*). Gaya memberikan batasan umum pada bentuk sistem.
- Pola Arsitektur (*Architectural Pattern*): Merupakan solusi spesifik, teruji, dan dapat berulang (*reusable solution*) untuk menyelesaikan masalah desain tertentu yang terjadi dalam konteks batasan gaya arsitektur tersebut (contoh: penerapan pola *Broker* untuk antrean pesan atau pola *MVC* untuk pemisahan antarmuka).

3.2.2 Penerapan Dua Pola Arsitektur dari Kategori Berbeda

Pola 1: Layered Architecture (Kategori: Structural Patterns)

- Alasan Penggunaan: Diterapkan untuk memisahkan tanggung jawab fungsional sistem Layanan Tugas Akhir secara tegas ke dalam lapisan-

lapisan logis: Lapisan Presentasi (*UI*), Lapisan Logika Bisnis (*Business Logic*), dan Lapisan Akses Data (*Data Access Layer*).

- Kelebihan:
    - Mempermudah pemeliharaan (*maintainability*) dan pengujian unit (*unit testing*) secara terisolasi pada masing-masing lapisan.
    - Mendukung pembagian kerja kelompok pengembang secara modular tanpa saling mengganggu kode di lapisan lain.
- **Harga yang Dibayar (Trade-off):**
    - Berpotensi menimbulkan penundaan waktu tanggap (*latency*) jika alur pemrosesan harus melewati terlalu banyak lapisan secara berurutan (*layers skipping risk*).

Pola 2: Publish-Subscribe / Message Broker Pattern (Kategori: Distributed / Messaging Patterns)

- Alasan Penggunaan: Diterapkan untuk mengatasi masalah utama berupa lonjakan trafik pendaftaran sidang secara serentak (*traffic spike*) serta mencegah terjadinya *race condition* dan *server crash*.
- Kelebihan:
    - Menjaga ketersediaan sistem (*high availability*) dan memastikan seluruh permintaan pendaftaran mahasiswa tertampung dengan aman tanpa ada data yang hilang.
    - Memisahkan proses penerimaan permintaan pengguna dari proses eksekusi berat di basis data (*asynchronous decoupling*).
- Kekurangan:
    - Menambah kompleksitas arsitektur sistem secara keseluruhan, sehingga membutuhkan pemantauan infrastruktur tambahan (*queue monitoring*) dan penanganan kegagalan pesan (*dead-letter queue*).

Bab 4

Desain Arsitektur

4.1 Desain Arsitektur dan Diagram(U5)

Bab ini menyajikan rujukan perancangan tingkat lanjut untuk subsistem *Assignment & Assessment* (Layanan Tugas Akhir UMI). Perancangan ini didasarkan pada prinsip-prinsip rekayasa perangkat lunak modern untuk menjamin skalabilitas, kemudahan pemeliharaan, serta integritas data akademik.

5.1 Prinsip Desain Arsitektur yang Diterapkan

Penerapan prinsip desain arsitektur pada subsistem Layanan Tugas Akhir dijabarkan melalui tiga pilar utama berikut:

1. **Separation of Concerns (SoC - Pemisahan Kepentingan):**
    - **Implementasi:** Sistem dibagi secara tegas menjadi tiga lapisan mandiri: **Lapisan Antarmuka (*Presentation Layer*)** untuk interaksi pengguna, **Lapisan Logika Bisnis (*Business Logic Layer*)** untuk aturan validasi SKS dan penjadwalan, serta **Lapisan Penyimpanan (*Data & Storage Layer*)** untuk basis data relasional dan *Object Storage*.
    - **Manfaat:** Memastikan perubahan pada tampilan antarmuka (misalnya pembaruan UI web/mobile) tidak akan merusak aturan validasi logika bisnis di backend.
2. **Single Responsibility Principle (SRP - Prinsip Tanggung Jawab Tunggal):**
    - **Implementasi:** Setiap modul, kelas, atau komponen layanan dirancang hanya memiliki satu alasan spesifik untuk berubah. Sebagai contoh, Modul Pendaftaran Sidang terisolasi secara murni untuk mengelola antrean pendaftaran, terpisah dari Modul Pengelolaan Berkas Skripsi (*File Uploader*) dan Modul Input Nilai Ujian.
    - **Manfaat:** Mengurangi *coupling* antar-modul dan memudahkan proses pengujian unit (*unit testing*) secara terisolasi.
3. **High Cohesion & Low Coupling (Kohesi Tinggi & Ketergantungan Rendah):**
    - **Implementasi:** Elemen-elemen fungsional yang saling berkaitan erat dikelompokkan ke dalam satu modul yang sama (*High Cohesion*), sementara komunikasi antar-modul dibatasi menggunakan antarmuka atau kontrak pesan yang terstandarisasi (*Low Coupling*).

- **Manfaat:** Meminimalkan dampak kerusakan (*ripple effect*) apabila terjadi perbaikan atau penambahan fitur baru pada masa mendatang.

4.2 Penjelasan Diagram Arsitektur

Diagram 1: Diagram Konteks & Aktor (*Use Case / Context Diagram*)

Deskripsi Fungsional & Batasan Sistem

Diagram konteks mendefinisikan batas sistem (*system boundary*) dari platform Layanan Tugas Akhir *Smart Campus* UMI. Sistem ini melayani tiga aktor utama yang memiliki hak akses dan batasan fungsional berbeda:

- **Mahasiswa Akhir:** Berperan sebagai pemrakarsa utama yang melakukan pengajuan judul, mengunggah draf skripsi (hingga 25 MB), serta mendaftarkan diri ke jadwal sidang.
- **Dosen Pembimbing & Penguji:** Berperan dalam mengontrol proses bimbingan berkala (*logbook*) serta memberikan evaluasi dan input nilai akhir ujian sidang.
- **Staf Administrasi Akademik:** Berperan sebagai verifikator administratif yang memvalidasi kelayakan berkas persyaratan sidang serta mengatur penjadwalan ruang dan dewan penguji.

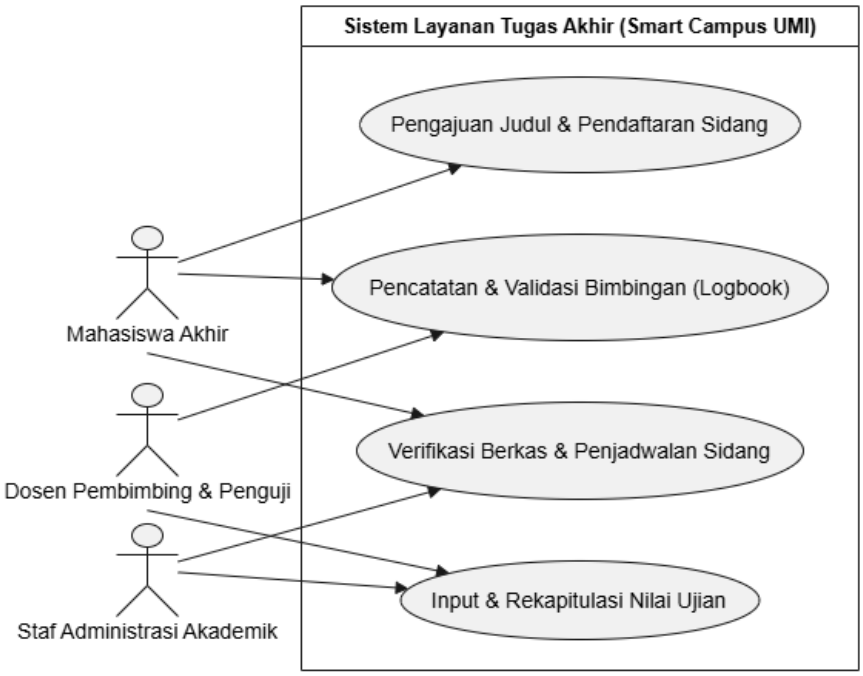


Diagram 2: Diagram Arsitektur Penerapan Pola (*Component & Connector / Broker Diagram*)

Penjelasan Komponen dan Alur Runtime

Diagram ini memetakan arsitektur saat sistem berjalan (*runtime architecture*) untuk menjawab tantangan *Performance Efficiency* (khususnya mengatasi lonjakan trafik / *traffic spike* pada jam pembukaan pendaftaran sidang).

- **Application Server (Web Server & Laravel):** Menerima permintaan HTTP awal dari pengguna. Web Server (Nginx) meneruskan *request* pendaftaran ke *backend* tanpa langsung mengeksekusi penulisan berat ke basis data secara sinkron.
- **Messaging Layer (RabbitMQ/Redis & Background Worker):** Permintaan pendaftaran ditampung ke dalam antrean (*queue*) pesan. *Background Worker* mengambil antrean secara bertahap untuk diproses secara asinkron (*asynchronous processing*), sehingga mencegah *server crash* dan *race condition*.
- **Database & Storage Layer (Relational DB & Object Storage):** Data transaksi pendaftaran disimpan ke basis data relasional, sedangkan berkas draf skripsi berukuran besar (25 MB) diunggah langsung ke *Cloud Object Storage* agar komputasi server utama tetap ringan.

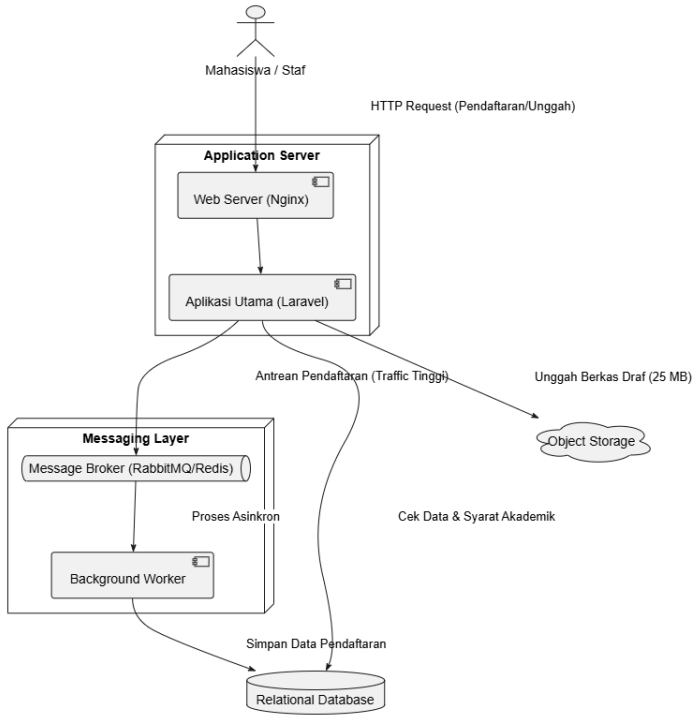
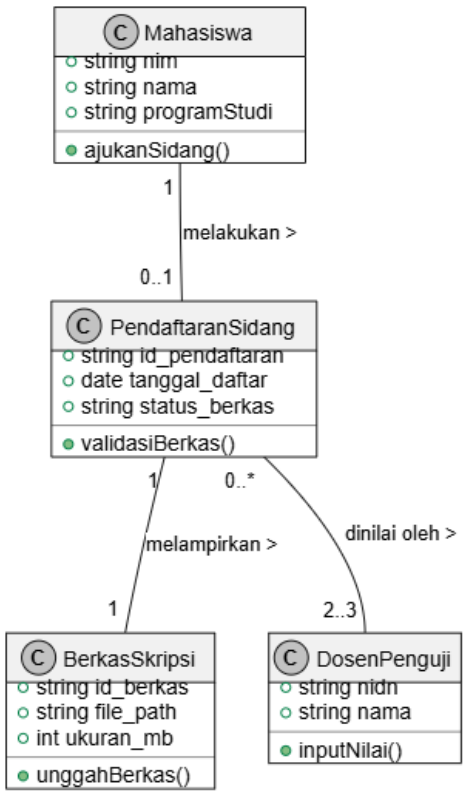


Diagram 3: Diagram UML Modul & Relasi Kelas (*Class Diagram*)

Struktur Data dan Hubungan Antar-Entitas

Diagram kelas ini menggambarkan pemodelan data struktural dari basis data relasional subsistem *Assignment & Assessment*:

- **Kelas Mahasiswa:** Memuat atribut identitas (nim, nama, programStudi) dan metode ajukanSidang(). Berelasi *1-to-0..1* dengan pendaftaran sidang.
- **Kelas PendaftaranSidang:** Entitas transaksional yang mencatat rekam pendaftaran (id_pendaftaran, tanggal_daftar, status_berkas) serta metode validasiBerkas().
- **Kelas BerkasSkripsi:** Berelasi *1-to-1* dengan pendaftaran sidang untuk merekam metadata berkas digital (id_berkas, file_path, ukuran_mb) dan metode unggahBerkas().
- **Kelas DosenPenguji:** Berelasi *0..*-to-*2..3* dengan pendaftaran sidang untuk merekam data dosen penguji (nidn, nama) serta metode inputNilai().



Bab 5

EVALUASI ARSITEKTUR DAN ADR

5.1 Evaluasi Arsitektur (ATAM & Utility Tree) (U6)

Bab ini menyajikan evaluasi kualitas arsitektur perangkat lunak menggunakan metode *Architecture Tradeoff Analysis Method* (ATAM) dan *Utility Tree* untuk subsistem Layanan Tugas Akhir (*Assignment & Assessment*). Evaluasi ini melibatkan umpan balik dari pemangku kepentingan utama, khususnya **Staf Administrasi Akademik**, guna memastikan sistem memenuhi atribut kualitas yang disyaratkan.

5.1.1 Utility Tree (Pohon Utilitas)

Utility Tree digunakan untuk menerjemahkan kebutuhan kualitas bisnis dan operasional akademik ke dalam skenario spesifik yang dapat diukur secara arsitektural.

Atribut Kualitas	**Skenario Stimulus**	**Respon yang Diharapkan**	**Taktik Arsitektur yang Diterapkan**
1. Performance Efficiency	Lonjakan 2.000 mahasiswa melakukan pendaftaran sidang dan unggah draf skripsi (25 MB) secara serentak.	Sistem tetap responsif, antrean pendaftaran tertampung tanpa *server crash*, dan waktu tanggap rata-rata di bawah 3 detik.	• *Message Broker (RabbitMQ/Redis)* • *Background Worker (Asynchronous)* • *Cloud Object Storage*
2. Security & Integrity	Upaya akses tidak sah atau manipulasi data nilai ujian oleh pihak luar/mahasiswa.	Seluruh komunikasi API terenkripsi, sesi pengguna diverifikasi secara ketat, dan integritas data nilai terlindungi.	• *JWT Authentication (JSON Web Token)* • *Role-Based Access Control (RBAC)*

Atribut Kualitas	Skenario Stimulus	Respon yang Diharapkan	Taktik Arsitektur yang Diterapkan
3. Maintainability	Pembaruan aturan validasi SKS atau penambahan modul verifikasi berkas oleh pengembang.	Modifikasi kode dapat dilakukan secara cepat tanpa merusak modul lain atau memicu *ripple effect*.	• *Layered Architecture* • *Single Responsibility Principle (SRP)*

5.1.2 Analisis Metode ATAM (*Architecture Tradeoff Analysis Method*)

1. Titik Sensitivitas (*Sensitivity Points*)

- **Penggunaan *Message Broker* pada Pendaftaran:** Menjadi titik sensitif utama terhadap atribut *Performance Efficiency*. Jika konfigurasi antrean (*queue*) tidak dioptimalkan, penumpukan data (*bottleneck*) dapat terjadi pada *worker*.
- **Pemisahan Lapisan (*Layered Architecture*):** Menjadi titik sensitif terhadap *Maintainability*. Keputusan memisahkan *Business Logic Layer* dari *Presentation Layer* memastikan perubahan logika akademik tidak memerlukan perombakan pada antarmuka pengguna.

2. Titik Pertukaran (*Tradeoff Points*)

- **Kompleksitas Keamanan vs. Kinerja Sistem:**
    - *Pertukaran:* Penerapan verifikasi token berlapis (*JWT & Session Check*) pada setiap permintaan unggah berkas besar (25 MB) menambah overhead komputasi jaringan.
    - *Solusi:* Kompleksitas ini diterima demi menjamin tingkat keamanan dan kerahasiaan dokumen akademik (*Security & Integrity*).
- **Asinkronisitas Pesan vs. Konsistensi Data Langsung:**
    - *Pertukaran:* Penggunaan *Message Broker* membuat status pendaftaran mahasiswa tidak langsung ditulis ke basis data utama pada detik yang sama (*eventual consistency*).

- *Solusi:* Sistem memberikan notifikasi status "Dalam Antrean (*Queued*)" kepada pengguna untuk menjaga transparansi pengalaman pengguna (*UX*).

3. Risiko dan Non-Risiko (*Risks & Non-Risks*)

- **Risiko (*Risks*):** Ketergantungan penuh pada layanan pihak ketiga atau *broker* eksternal (seperti Redis/RabbitMQ). Jika layanan tersebut mengalami kegagalan (*outage*), proses pendaftaran sidang dapat tertunda sementara.
- **Non-Risiko (*Non-Risks*):** Desain modular (*split module*) pada subsistem penilaian terbukti aman dan tidak menimbulkan dampak negatif terhadap modul bimbingan (*logbook*) yang berjalan secara paralel.

5.2 Architecture Decision Records (ADR) (U7)

Dokumen Architectural Decision Record (ADR) ini mencatat keputusan-keputusan arsitektur penting yang ditetapkan dalam perancangan Layanan Tugas Akhir pada *subsistem Assignment & Assessment* Universitas Muslim Indonesia (UMI). Setiap ADR memuat konteks permasalahan, keputusan yang dipilih, serta konsekuensi teknis yang ditimbulkan berdasarkan kebutuhan dan hasil evaluasi arsitektur.

ADR 001: Penggunaan *Message Broker* untuk Penanganan Lonjakan Trafik Pendaftaran Sidang

- **Status:** Diterima (*Accepted*)
- **Konteks:** Pada periode pembukaan pendaftaran sidang akhir semester, sistem mengalami lonjakan trafik (*traffic spike*) secara masif di mana ribuan mahasiswa mengakses server secara bersamaan untuk mengunggah draf skripsi (ukuran berkas hingga 25 MB) dan mengirimkan data pendaftaran. Pendekatan sinkron langsung ke basis data berisiko tinggi menyebabkan *database bottleneck*, *race condition*, dan kegagalan server (*server crash*).
- **Keputusan:** Menerapkan **Message Broker Pattern** menggunakan antrean pesan (*RabbitMQ/Redis Queue*) yang dikombinasikan dengan *Background Worker* untuk memproses pengajuan pendaftaran dan unggahan berkas secara asinkron.
- **Konsekuensi:**

- **Positif:** Sistem menjadi jauh lebih tahan banting (*resilient*) terhadap lonjakan beban, menekan tingkat kegagalan (*error rate*) hingga mendekati 0%, serta menjaga waktu tanggap server tetap cepat.
- **Negatif / Tantangan:** Menambahkan kompleksitas infrastruktur operasional (*deployment* dan pemantauan layanan *broker*). Status pendaftaran mahasiswa menjadi bersifat konsisten secara bertahap (*eventual consistency*), sehingga memerlukan indikator status "Dalam Antrean" pada antarmuka pengguna.

ADR 002: Penerapan *JWT Authentication* dan *Role-Based Access Control (RBAC)*

- **Status:** Diterima (*Accepted*)
- **Konteks:** Subsistem Layanan Tugas Akhir melibatkan multi-pengguna dengan hak akses yang sangat sensitif, mulai dari pengunggahan draf dokumen mahasiswa, validasi berkas oleh Staf Administrasi Akademik, hingga penginputan nilai ujian oleh Dosen Penguji. Sistem memerlukan mekanisme autentikasi yang aman, *stateless*, dan terenkripsi untuk mencegah manipulasi data dan akses tidak sah.
- **Keputusan:** Menggunakan **JWT (JSON Web Token)** untuk autentikasi sesi pengguna berbasis API serta menerapkan **Role-Based Access Control (RBAC)** secara ketat guna membatasi hak akses berdasarkan peran aktor (Mahasiswa, Dosen, dan Staf Administrasi).
- **Konsekuensi:**
  - **Positif:** Memastikan integritas dan kerahasiaan data akademik terlindungi dengan baik, serta memudahkan pengelolaan skalabilitas sesi pengguna tanpa membebani memori server pusat (*stateless*).
  - **Negatif / Tantangan:** Memerlukan penanganan kedaluwarsa token (*token expiration*) dan mekanisme *refresh token* yang cermat agar tidak mengganggu kenyamanan pengguna saat berinteraksi dengan sistem.

ADR 003: Penerapan *Layered Architecture* dan *Split Module*

- **Status:** Diterima (*Accepted*)
- **Konteks:** Seiring bertambahnya fitur di masa mendatang (seperti penambahan jenis evaluasi baru atau integrasi sistem akademik universitas

lainnya), kode sumber subsistem harus mudah dipelihara (*maintainable*) dan dikembangkan oleh tim pengembang yang berbeda tanpa memicu kerusakan pada modul yang sudah berjalan.

- **Keputusan:** Menerapkan pola **Layered Architecture** (Pemisahan tegas antara *Presentation Layer*, *Business Logic Layer*, dan *Data Layer*) serta memecah fungsionalitas ke dalam modul-modul mandiri (*Split Module* / SRP).
- **Konsekuensi:**
    - **Positif:** Mempermudah proses pengujian unit (*unit testing*), mengurangi dampak kerusakan turunan (*ripple effect*) saat ada perubahan kode, dan meningkatkan keterbacaan kode sumber (*maintainability*).
    - **Negatif / Tantangan:** Menuntut kedisiplinan tim pengembang yang tinggi dalam menjaga batas-batas arsitektur (*layer boundaries*) agar tidak terjadi ketergantungan silang yang melanggar prinsip modularitas.

Bab 6

Penutup

6.1 Kesimpulan

Berdasarkan hasil analisis dan perancangan yang telah dilakukan, Layanan Tugas Akhir pada subsistem Assignment & Assessment Universitas Muslim Indonesia (UMI) membutuhkan arsitektur yang mampu mendukung proses akademik secara lebih efektif, aman, dan terstruktur.

Hasil wawancara menunjukkan beberapa kendala dalam pelaksanaan layanan, seperti lambatnya akses ketika banyak mahasiswa melakukan pendaftaran secara bersamaan, proses unggah dokumen yang terkadang mengalami kegagalan, serta kendala dalam pengelolaan bimbingan, pemeriksaan berkas, dan penjadwalan sidang. Permasalahan tersebut menjadi dasar dalam menentukan tiga atribut kualitas utama, yaitu Performance Efficiency, Security & Integrity, dan Maintainability.

Untuk memenuhi kebutuhan tersebut, rancangan menggunakan Layered Architecture untuk memisahkan fungsi antarlapisan serta Message Broker dan Background Worker untuk membantu menangani lonjakan permintaan.

Selain itu, penggunaan JWT dan Role-Based Access Control (RBAC) diterapkan untuk membatasi akses pengguna sesuai dengan perannya masing-masing.

Hasil evaluasi menggunakan metode ATAM menunjukkan bahwa rancangan memiliki beberapa pertimbangan, terutama pada hubungan antara keamanan dan kinerja serta proses asynchronous dan konsistensi data. Keputusan arsitektur yang telah dibuat diharapkan dapat membantu meningkatkan kestabilan layanan, menjaga keamanan informasi akademik, dan mempermudah pengembangan pada masa mendatang.

6.2 Saran

Berdasarkan hasil perancangan, terdapat beberapa hal yang dapat diperhatikan untuk pengembangan selanjutnya, yaitu:

1. Melakukan pengujian kinerja untuk memastikan layanan mampu menangani peningkatan jumlah pengguna, terutama pada periode pendaftaran seminar dan sidang.
2. Meningkatkan pengelolaan logbook dan pemberitahuan otomatis agar mahasiswa serta dosen dapat memantau perkembangan bimbingan dengan lebih mudah.
3. Mengembangkan validasi dokumen secara otomatis untuk mengurangi pemeriksaan berulang terhadap berkas yang tidak lengkap atau tidak sesuai format.
4. Menyempurnakan pengaturan jadwal sidang dan pemantauan kuota dosen pembimbing agar tidak terjadi benturan jadwal maupun pembagian beban yang tidak merata.
5. Melakukan pemantauan terhadap Message Broker, keamanan akses, dan penyimpanan data secara berkala agar layanan tetap berjalan dengan baik.

Dengan adanya pengembangan dan evaluasi secara berkelanjutan, diharapkan Layanan Tugas Akhir dapat memberikan kemudahan bagi mahasiswa, dosen, dan staf akademik dalam menjalankan seluruh proses penyelesaian tugas akhir.

DAFTAR PUSTAKA

L. Bass, P. Clements, and R. Kazman, Software Architecture in Practice, 4th ed. Boston, MA, USA: Addison-Wesley Professional, 2021.

R. Kazman, M. Klein, and P. Clements, "ATAM: Method for Architecture Evaluation," Software Engineering Institute, Carnegie Mellon University, Pittsburgh, PA, USA, Tech. Rep. CMU/SEI-2000-TR-004, Aug. 2000.

R. C. Martin, Clean Architecture: A Craftsman's Guide to Software Structure and Design. Boston, MA, USA: Prentice Hall, 2017.

M. T. Nygard, Release It!: Design and Deploy Production-Ready Software, 2nd ed. Raleigh, NC, USA: Pragmatic Bookshelf, 2018.

S. Newman, Building Microservices: Designing Fine-Grained Systems, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2021.

M. Richards and N. Ford, Fundamentals of Software Architecture: An Engineering Approach. Sebastopol, CA, USA: O'Reilly Media, 2020.